

# 2 Exact and Approximate Dynamic Programming

## Learning objectives

After studying this chapter, the reader should be able to:

1. formulate deterministic and stochastic finite-horizon control problems with the correct policy and information structure;
2. state Bellman's principle of optimality and derive finite-horizon DP algorithms in value and Q-factor form;
3. formulate stationary infinite-horizon problems and explain the roles of Bellman operators, VI, and PI under the discounted reference assumptions;
4. distinguish exact DP from approximate DP by identifying whether an approximation enters the control problem, cost-to-go, conditional expectation, or feasible-control minimization; and
5. reformulate a finite-horizon Markov problem as a stationary absorbing total-cost problem and distinguish this exact reformulation from repeatedly deploying a finite-horizon controller on a continuing objective.

## Chapter roadmap

The chapter turns the Bellman lens introduced in Chapter 1 into a reference set of exact DP computations and then identifies where approximate DP replaces them by tractable surrogates. Sections 2.1–2.3 develop deterministic and stochastic finite-horizon backward DP; Sections 2.4–2.5 introduce finite-horizon approximation and the HEV study. Sections 2.6–2.8 then develop stationary infinite-horizon models, exact VI and PI, and their approximate structures as a bridge to Chapter 4. Section 2.9 gives a general finite-to-infinite-horizon reformulation. Section 2.10 uses the lane-keeping study first to verify that finite-horizon backward DP and its augmented infinite-horizon representation are equivalent, and then to contrast repeatedly deployed finite-horizon control with a genuinely stationary infinite-horizon design. Figure 2.1 summarizes this progression.

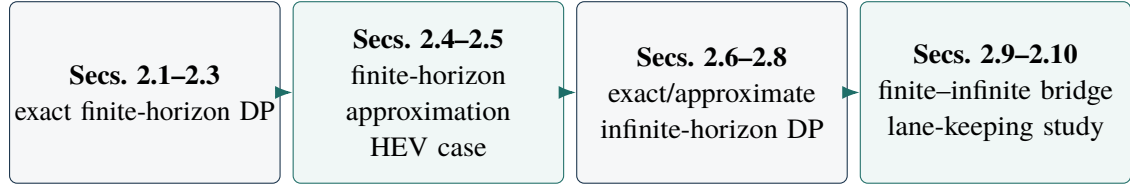


Figure 2.1: Roadmap of Chapter 2. Finite-horizon recursion makes the principle of optimality explicit; stationary infinite-horizon operators then provide the main reusable reference. Approximate DP replaces selected exact objects or operations, while the final study distinguishes an exact reformulation from a genuine change of control horizon.

## 2.1 From Bellman’s Principle to Dynamic Programming

Chapter 1 introduced Bellman’s principle of optimality as this book’s organizing lens, while emphasizing that it is not the only route to optimal control. This chapter first turns that principle into exact finite- and infinite-horizon DP algorithms for a declared model, and then shows how approximate DP replaces selected problems, objects, or computations by tractable surrogates. The difficulty is not merely that a decision changes the next state. A current control changes the feasible and economically attractive decisions that remain, so optimizing each stage in isolation can spend energy, authority, or safety margin that has greater value later. DP captures this intertemporal tradeoff by assigning a value to every remaining tail problem and reusing those values when current controls are compared.

Finite-horizon DP comes first because its terminal boundary, stage-indexed costs-to-go  $J_k$ , and one backward pass are transparent. The chapter then turns to the infinite-horizon setting, which is the main reference for many continuing control problems. When the dynamics, cost, and information rules do not change with the absolute stage, returning to the same state presents the same remaining control problem. The stage-indexed family  $J_k$  can then be replaced by one reusable value function  $J$ , and the associated feedback rule can likewise be reused as a stationary policy  $\mu$ . Under assumptions stated later, this recurring structure is characterized by a Bellman fixed-point equation and supports VI, PI, approximate DP, RL, and online policy improvement.

Later sections of this chapter introduce two representative ways to make such an infinite-horizon problem mathematically well posed. A discounted continuing formulation gives progressively less weight to costs farther in the future, whereas a stochastic shortest-path (SSP) formulation ends when an absorbing terminal state is reached. These details are developed only after finite-horizon DP has established the basic Bellman logic. Finite-horizon models nevertheless remain essential for trips, missions, scheduling, episodic tasks, rollout, and MPC. A later section gives a general reformulation that adds the stage or remaining horizon and an absorbing terminal state. It represents the original finite-horizon objective exactly as a stationary infinite-horizon total-cost problem; it does not replace that objective by a different continuing-control objective.

Bellman's principle leads to the central computational relation

$$\text{optimal cost-to-go now} = \min_{\text{feasible current control}} \{ \text{current cost} + \text{optimal cost-to-go from the successor} \}. \quad (2.1)$$

Equation (2.1) first characterizes how an optimal current decision must account for the remaining problem. It becomes a computational procedure only if the cost-to-go can be represented and the current cost, successor cost-to-go, and feasible-control minimization can be evaluated; a stochastic problem additionally requires a conditional expectation over possible successors. This distinction leads to two questions that remain separate throughout the book:

1. *Characterization*: what Bellman equation do the optimal cost-to-go and feedback policy satisfy for the declared problem?
2. *Computation*: how are the objects in that equation represented, and how are its expectation and minimization actually evaluated?

The characterization question supplies the exact reference against which an implementation is judged. Accordingly, *exact DP* in this chapter means that every operation in the relevant Bellman equation is performed without approximation for the declared mathematical or numerical model; it does not assert that the model perfectly describes the physical plant. *Approximate DP* instead modifies the problem or replaces its cost-to-go, conditional expectation, or feasible-control minimization by a tractable surrogate. Discretization, fitted representations, and sample-based backups are different ways of making those replacements. Where the resulting computation is performed is a further, separate choice. In particular, *online planning* uses a fixed model together with a fixed value or tail approximation to solve a current-state optimization during deployment.

With these distinctions in place, the chapter uses general state-space formulations where possible, finite tables where they make exact algorithms and convergence especially transparent, and linear–quadratic (LQ) examples to show that DP is not intrinsically tabular. The finite-horizon and stationary infinite-horizon formulations follow the optimal-control treatments in Bertsekas (2019) and Bertsekas (2025). When specialized to a finite Markov decision process (MDP), the discounted notation also aligns with Sutton and Barto (2018).

## 2.2 Deterministic Finite-Horizon Dynamic Programming

A finite-horizon problem has  $N \geq 1$  control stages. Consider the deterministic system

$$x_{k+1} = f_k(x_k, u_k), \quad k = 0, \dots, N-1, \quad (2.2)$$

where  $x_k \in \mathcal{X}_k$  is the state,  $u_k \in \mathcal{U}_k(x_k)$  is an admissible control,  $g_k(x_k, u_k)$  is the stage cost, and  $g_N(x_N)$  is the terminal cost. Stage  $k$  is one decision–transition–cost step: the controller observes  $x_k$ , selects  $u_k$ , incurs  $g_k$ , and reaches  $x_{k+1}$ . The map  $f_k$  is the *system dynamics*, or equivalently the *state-transition model*. The former phrase is customary in control; the latter is common in MDP and RL descriptions and extends naturally to a stochastic transition law. A realized step  $x_k \xrightarrow{u_k} x_{k+1}$  together with its incurred cost is a *transition sample*. A stage need not

equal one physical second; it may represent a sample interval, route segment, task assignment, or event. This timing is shown in Figure 2.2.

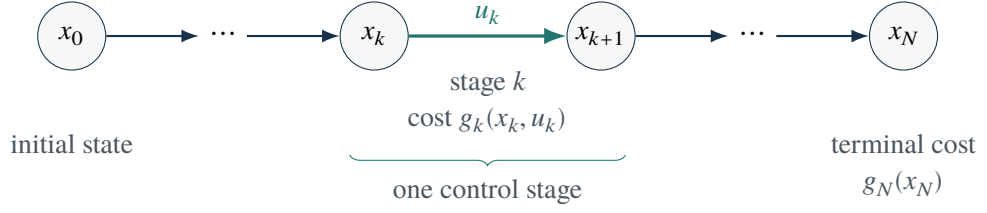


Figure 2.2: A deterministic finite-horizon decision process. Stage  $k$  starts at  $x_k$  and ends at  $x_{k+1} = f_k(x_k, u_k)$  after control  $u_k$  and stage cost  $g_k(x_k, u_k)$ . The terminal cost is incurred after the final transition.

For a prescribed initial state  $x_0$ , the cost of a control sequence is

$$J(x_0; u_0, \dots, u_{N-1}) = \sum_{k=0}^{N-1} g_k(x_k, u_k) + g_N(x_N). \quad (2.3)$$

Let  $\mathcal{U}_{k:N-1}(x_k)$  denote the set of control tails  $(u_k, \dots, u_{N-1})$  that start from the specified  $x_k$  and satisfy the dynamics and the stagewise constraints  $u_t \in \mathcal{U}_t(x_t)$ . The optimal value of the original problem is then

$$J^*(x_0) := \min_{(u_0, \dots, u_{N-1}) \in \mathcal{U}_{0:N-1}(x_0)} J(x_0; u_0, \dots, u_{N-1}). \quad (2.4)$$

The terminal cost  $g_N$  may encode a desired final state, a residual value, a soft terminal penalty, or a hard terminal requirement through the extended value  $+\infty$ .

At stage  $k$ , define the *optimal cost-to-go*—the optimal cost of the tail subproblem that starts from  $(k, x_k)$ —by

$$J_k^*(x_k) := \min_{(u_k, \dots, u_{N-1}) \in \mathcal{U}_{k:N-1}(x_k)} \left\{ \sum_{t=k}^{N-1} g_t(x_t, u_t) + g_N(x_N) \right\}, \quad x_k \in \mathcal{X}_k. \quad (2.5)$$

In particular,  $J^*(x_0) = J_0^*(x_0)$ . A finite-horizon state-feedback policy is a sequence  $\mu = (\mu_0, \dots, \mu_{N-1})$  with  $\mu_k(x_k) \in \mathcal{U}_k(x_k)$ . It is generally time dependent: the control can depend on both the current state and the stage.

A policy  $u_k = \mu_k(x_k)$  is often called a *nonstationary Markov policy*. Here “Markov” means that the current state summarizes the relevant past, so no additional history is needed once  $x_k$  is known. “Nonstationary” means that the feedback map may change with  $k$  even when the same physical state recurs, because the remaining horizon and future data have changed. These properties are independent: a policy can be Markov and nonstationary. By contrast, in the stationary infinite-horizon problems developed later, the model and remaining problem repeat, so one stage-independent Markov rule  $u = \mu(x)$  can be optimal under the assumptions stated there. A nonstationary finite-horizon sequence can nevertheless be written as one stationary rule after the stage is included in the augmented state  $z_k = (k, x_k)$ ; a later section develops that reformulation carefully.

The principle below concerns an attained optimum, so a minimal well-posedness condition is stated explicitly. Allowing the extended value  $+\infty$  keeps hard terminal constraints compatible with the recursion. The condition is automatic in the elementary finite tables used later when every feasible set is nonempty, but it also allows continuous spaces under the usual existence conditions.

**Assumption 2.1** (Well-posed deterministic finite-horizon problem). Stage and terminal costs may take values in  $\mathbb{R} \cup \{+\infty\}$ . Every feasible-control set is nonempty and, for every  $x_k \in \mathcal{X}_k$  and  $u_k \in \mathcal{U}_k(x_k)$ ,  $f_k(x_k, u_k) \in \mathcal{X}_{k+1}$ . The displayed extended-value minima are attained, and the prescribed initial state admits at least one finite-cost control sequence. Finite state and control sets satisfy the attainment requirement automatically; continuous spaces require the corresponding regularity and existence conditions.

Bellman's principle is a structural statement about any optimal trajectory; it precedes the recursion used to compute such a trajectory (Bertsekas 2019, Sec. 1.1).

**Proposition 2.1** (Bellman's principle of optimality). *Suppose assumption 2.1 holds. If a control sequence is optimal from  $x_0$  and reaches  $x_k^*$  at stage  $k \in \{0, \dots, N-1\}$ , then its remaining controls  $(u_k^*, \dots, u_{N-1}^*)$  are optimal for the tail problem beginning at  $(k, x_k^*)$ :*

$$(u_k^*, \dots, u_{N-1}^*) \in \arg \min_{(u_k, \dots, u_{N-1}) \in \mathcal{U}_{k:N-1}(x_k^*)} \left\{ \sum_{t=k}^{N-1} g_t(x_t, u_t) + g_N(x_N) \right\}, \quad x_k = x_k^*. \quad (2.6)$$

*Proof.* If the remaining controls were not optimal for the tail problem, replacing them by a feasible tail of strictly smaller cost would leave the prefix through  $x_k^*$  unchanged and reduce the total cost. This contradicts optimality of the original sequence.  $\square$

Bellman's principle concerns the tail encountered along an optimal trajectory. DP turns it into a computation by applying the same decomposition at every stage–state pair. Fix an arbitrary  $x_k \in \mathcal{X}_k$ . Each feasible current control  $u_k$  produces an immediate cost, a successor  $x_{k+1}$ , and a tail subproblem. If the continuation from that successor were not optimal, it could be replaced by one with lower cost. Hence the best tail beginning with  $u_k$  has cost  $g_k(x_k, u_k) + J_{k+1}^*(x_{k+1})$ , and minimizing these candidate costs gives  $J_k^*(x_k)$ . Applying this calculation backward from the terminal stage is the finite-horizon *Bellman recursion*.

Evaluating the right-hand side of a Bellman recursion at one stage–state pair—including, in a stochastic problem, the conditional expectation for each feasible current control, followed by minimization over those controls—is called a *Bellman backup*. This book calls a minimizing control selected by that local comparison a *Bellman decision*. Figure 2.3 juxtaposes the structural principle with the computation it motivates.

The upper panel concerns a tail that occurs on an optimal trajectory; the lower panel applies the same decomposition to arbitrary declared states, including states that the final trajectory may not visit. The next proposition states this recursion, and Algorithm 2.1 separates its backward computation from forward trajectory reconstruction.

**Proposition 2.2** (Deterministic finite-horizon Bellman recursion). *Under assumption 2.1, initialize*

$$J_N^*(x_N) = g_N(x_N), \quad \text{for every } x_N \in \mathcal{X}_N. \quad (2.7)$$

*Then, for  $k = N - 1, \dots, 0$ ,*

$$J_k^*(x_k) = \min_{u_k \in \mathcal{U}_k(x_k)} \{g_k(x_k, u_k) + J_{k+1}^*(f_k(x_k, u_k))\}, \quad \text{for every } x_k \in \mathcal{X}_k. \quad (2.8)$$

*Any selector*

$$u_k^* \in \arg \min_{u_k \in \mathcal{U}_k(x_k)} \{g_k(x_k, u_k) + J_{k+1}^*(f_k(x_k, u_k))\}, \quad x_k \in \mathcal{X}_k, \quad (2.9)$$

*defines an optimal time-dependent state-feedback policy, and the value returned at the initial stage satisfies  $J_0^*(x_0) = J^*(x_0)$ .*

*Proof.* At  $k = N$ , no control remains, so the terminal cost is the entire cost-to-go and Equation (2.7) holds. Fix  $k < N$  and an arbitrary  $x_k \in \mathcal{X}_k$ , and suppose that  $J_{k+1}^*$  is the optimal cost-to-go function at stage  $k + 1$  for every feasible successor state. Every feasible tail from  $x_k$  decomposes into a first control  $u_k$  and a continuation from  $x_{k+1} = f_k(x_k, u_k)$ . Among all feasible tails whose first control is  $u_k$ , an optimal continuation must solve the successor tail problem and therefore has cost

$$g_k(x_k, u_k) + J_{k+1}^*(f_k(x_k, u_k)).$$

Conversely, concatenating any feasible  $u_k$  with an optimal continuation from its successor produces a feasible tail with exactly this cost. Minimizing over the feasible first controls proves Equation (2.8); selecting a minimizer proves Equation (2.9). Backward induction from  $N$

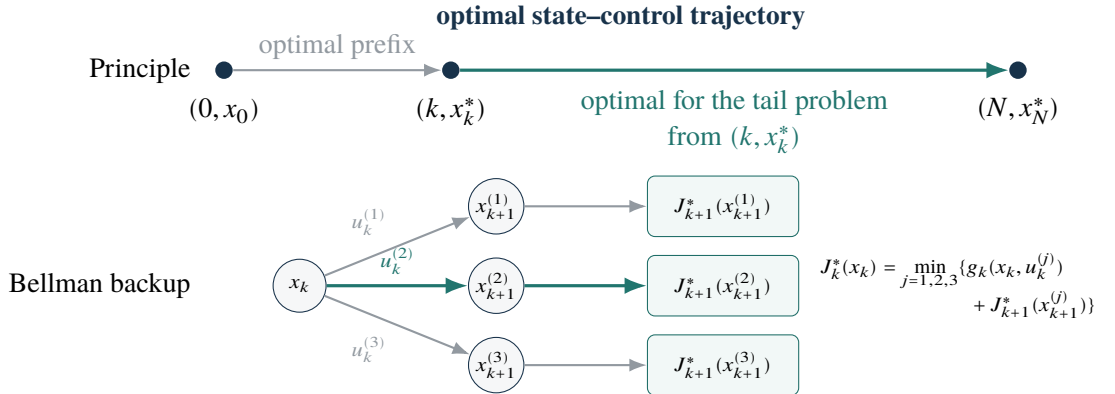


Figure 2.3: Bellman's principle and the local computation it justifies. The upper panel identifies the tail of an optimal trajectory as a solution of its tail subproblem. The lower panel shows one Bellman backup at an arbitrary  $x_k$ ; the three displayed controls comprise the feasible candidate set in this schematic, and  $x_{k+1}^{(j)} = f_k(x_k, u_k^{(j)})$ .

establishes the result at every stage. At  $k = 0$ , the tail problem is the original problem, so  $J_0^*(x_0) = J^*(x_0)$ .  $\square$

### Algorithm 2.1: Deterministic finite-horizon DP

For a finite-state implementation in which each feasible-control minimization can be evaluated exactly:

1. **Backward initialization:** set  $J_N^*(x_N) = g_N(x_N)$  for every  $x_N \in \mathcal{X}_N$ .
2. **Backward recursion:** for  $k = N - 1, \dots, 0$ , and for every  $x_k \in \mathcal{X}_k$ , compute

$$J_k^*(x_k) = \min_{u_k \in \mathcal{U}_k(x_k)} \{g_k(x_k, u_k) + J_{k+1}^*(f_k(x_k, u_k))\}.$$

3. **Forward initialization:** for the prescribed initial state, compute

$$u_0^* \in \arg \min_{u_0 \in \mathcal{U}_0(x_0)} \{g_0(x_0, u_0) + J_1^*(f_0(x_0, u_0))\}, \quad x_1^* = f_0(x_0, u_0^*).$$

4. **Forward continuation:** for  $k = 1, \dots, N - 1$ , use

$$u_k^* \in \arg \min_{u_k \in \mathcal{U}_k(x_k^*)} \{g_k(x_k^*, u_k) + J_{k+1}^*(f_k(x_k^*, u_k))\}, \quad x_{k+1}^* = f_k(x_k^*, u_k^*).$$

Return  $J_0^*(x_0)$  and the reconstructed optimal state–control trajectory.

When the full model is available in advance, the backward pass is ordinarily offline and stores the exact cost-to-go functions  $J_k^*$ . The forward minimizations in Algorithm 2.1 then recover the optimal controls  $u_k^*$  along the trajectory from the prescribed  $x_0$ . Equivalently, one may store the minimizing selectors  $\mu_k^*$  in Equation (2.9) during the backward pass and apply  $u_k^* = \mu_k^*(x_k^*)$ ; evaluating this stored feedback law is policy execution, not online planning. If instead only  $J_{k+1}^*$  is stored and the displayed minimization is solved from the measured state during deployment, the controller performs one-step online planning in the restricted computational sense used in this book: the model and cost-to-go function remain fixed, while a current-state optimization is performed online. With the exact model, exact  $J_{k+1}^*$ , exact minimization, and the same deterministic tie-breaking rule, the two implementations return the same optimal control; the online minimization recovers, rather than improves upon, the exact DP policy. Resolving a longer remaining tail problem instead constitutes multistep online planning.

The same exact recursion can be written with a state–control value, or *Q-factor*. For  $k = 0, \dots, N - 1$  and every admissible  $(x_k, u_k)$ , define

$$Q_k^*(x_k, u_k) = g_k(x_k, u_k) + J_{k+1}^*(f_k(x_k, u_k)), \quad x_k \in \mathcal{X}_k, \quad u_k \in \mathcal{U}_k(x_k). \quad (2.10)$$

Then, for every  $x_k \in \mathcal{X}_k$  and  $k = 0, \dots, N - 1$ ,

$$J_k^*(x_k) = \min_{u_k \in \mathcal{U}_k(x_k)} Q_k^*(x_k, u_k), \quad \mu_k^*(x_k) \in \arg \min_{u_k \in \mathcal{U}_k(x_k)} Q_k^*(x_k, u_k). \quad (2.11)$$

The Q-factor-only backward recursion starts at the last decision stage with

$$Q_{N-1}^*(x_{N-1}, u_{N-1}) = g_{N-1}(x_{N-1}, u_{N-1}) + g_N(f_{N-1}(x_{N-1}, u_{N-1})).$$

For  $k = N - 2, \dots, 0$ , and every admissible  $(x_k, u_k)$ , let  $x_{k+1} = f_k(x_k, u_k)$ . Eliminating  $J_{k+1}^*$  then gives

$$Q_k^*(x_k, u_k) = g_k(x_k, u_k) + \min_{v \in \mathcal{U}_{k+1}(x_{k+1})} Q_{k+1}^*(x_{k+1}, v). \quad (2.12)$$

Thus  $J_k^*$  ranks states before the current control is selected, whereas  $Q_k^*$  ranks a particular current control together with its best continuation.

If stage  $k$  has  $n_k$  states and at most  $m_k$  controls per state, the complete tabular backward pass evaluates

$$C_{\text{DP}} = \sum_{k=0}^{N-1} \sum_{x \in \mathcal{X}_k} |\mathcal{U}_k(x)| \leq \sum_{k=0}^{N-1} n_k m_k$$

Bellman candidates. By contrast, enumerating all open-loop control sequences for one initial state can require as many as  $\prod_{k=0}^{N-1} m_k$  complete trajectories, each extending across the horizon. For example, if there are  $n$  states at each of the  $N$  decision stages and  $n$  feasible controls at every state, and if one candidate transition can be evaluated in constant time, tabular DP requires  $O(Nn^2)$  candidate evaluations. Brute-force enumeration can require evaluating  $n^N$  complete control sequences. DP gains efficiency by solving a repeated tail state once and reusing its value.

However, DP does not eliminate the *curse of dimensionality*: its table can itself become prohibitively large as the state and control dimensions increase. If a  $d$ -dimensional state is discretized with  $q$  values per coordinate and a  $p$ -dimensional control with  $r$  values per coordinate, then  $n = q^d$  and  $m = r^p$ , so a direct backward pass scales as  $O(Nq^d r^p)$  for the deterministic backup above. This growth is one principal reason approximation is needed in many problems; exploitable analytic structure can sometimes avoid a grid, as the LQ example below demonstrates.

### Finite-state example: single-vehicle service-request scheduling.

Consider one mobility-service vehicle that must serve requests  $A$ ,  $B$ , and  $C$  over stages 0, 1, and 2. The state  $S_k$  is the set of requests already completed, with  $S_0 = \emptyset$ , and the control selects the next unserved request:

$$u_k \in \{A, B, C\} \setminus S_k, \quad S_{k+1} = S_k \cup \{u_k\}.$$

Each entry below is a normalized incremental penalty combining the stage-dependent delay and operating cost of serving that request next. Assume the vehicle returns to the same depot after completing every request. Its physical departure location is therefore identical at the start of every stage and need not be included in  $S_k$ ; the displayed costs have already been computed for departures from this common hub:

|         | $A$ | $B$ | $C$ |
|---------|-----|-----|-----|
| $k = 0$ | 3   | 2   | 6   |
| $k = 1$ | 4   | 5   | 1   |
| $k = 2$ | 2   | 3   | 4   |

with zero terminal cost. Thus  $g_k(S_k, u_k)$  is the corresponding stage-and-request entry in the table; for every reachable state,  $k = |S_k|$ , while  $S_k$  determines which requests remain feasible. This is a shortest-path problem on a directed acyclic state graph: each arc adds one served request, and the path cost is the sum of its arc costs. Figure 2.4 displays the full state graph, the optimal remaining cost at each node, and the optimal path.

If a vehicle instead remains at the previous request's destination, its current location must be added to the state. Extending the setting to a fleet also requires each vehicle's location and availability, together with the vehicle–request assignment decisions. The reduced single-vehicle state here is chosen to expose backward DP, not to serve as a complete dispatch model.

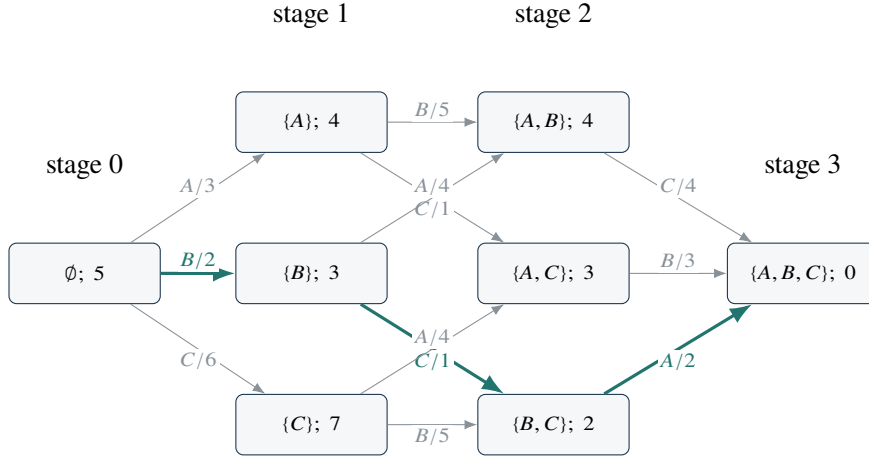


Figure 2.4: Backward DP for scheduling three mobility-service requests. A node displays “completed requests; optimal remaining cost,” and an arc label displays “next request/incremental cost.” The highlighted path serves  $B, C, A$  and has  $J^*(\emptyset) = 5$ .

The numbers displayed at the nodes are optimal remaining costs, not additional stage costs. The backward pass follows Algorithm 2.1:

$$\begin{aligned}
 J_3^*({A, B, C}) &= 0, \\
 J_2^*({A, B}) &= 4, \quad J_2^*({A, C}) = 3, \quad J_2^*({B, C}) = 2, \\
 J_1^*({A}) &= \min\{5 + 4, 1 + 3\} = 4, \\
 J_1^*({B}) &= \min\{4 + 4, 1 + 2\} = 3, \\
 J_1^*({C}) &= \min\{4 + 3, 5 + 2\} = 7, \\
 J_0^*(\emptyset) &= \min\{3 + 4, 2 + 3, 6 + 7\} = 5.
 \end{aligned}$$

Following the stored minimizers forward yields the service order  $B, C, A$ . For  $n$  requests, brute-force enumeration considers  $n!$  complete service orders. In subset DP, stage  $k$  contains  $\binom{n}{k}$  completed-request sets, and each such state evaluates  $n - k$  remaining requests. Hence the total number of Bellman candidates is

$$\sum_{k=0}^{n-1} \binom{n}{k} (n - k) = n 2^{n-1}.$$

For sufficiently large  $n$ , the factorial count  $n!$  outgrows  $n 2^{n-1}$ , and the computational advantage of DP increases. The reason is reuse: different orders and their accumulated costs are still compared, but once their histories reach the same completed-request set, the remaining cost-to-go is computed only once. Nevertheless,  $n 2^{n-1}$  is still exponential in  $n$ , so DP reduces—but does not eliminate—the rapid growth in computation.

### Continuous-state and continuous-control example: finite-horizon LQ control.

Continuous spaces do not always require discretization. A principal exact case is unconstrained finite-horizon LQ control: linear dynamics and quadratic costs are closed under the Bellman recursion, so every cost-to-go remains quadratic and every minimization can be performed analytically. Let  $x_k \in \mathbb{R}^n$  and  $u_k \in \mathbb{R}^m$ , and consider the LQ problem

$$x_{k+1} = A_k x_k + B_k u_k, \quad J(x_0; u_0, \dots, u_{N-1}) = x_N^\top Q_N x_N + \sum_{k=0}^{N-1} (x_k^\top Q_k x_k + u_k^\top R_k u_k), \quad (2.13)$$

where  $\mathcal{U}_k(x_k) = \mathbb{R}^m$ ,  $Q_k \geq 0$ ,  $Q_N \geq 0$ , and  $R_k > 0$ . Here  $Q_k$  is a state-cost matrix, not the Q-factor introduced above. The terminal cost is quadratic, and substituting a quadratic successor cost into the Bellman recursion produces a quadratic expression in the current state and control. Minimizing that expression leaves another quadratic function of the current state; this is the backward-induction argument. At the terminal stage,  $J_N^*(x_N) = x_N^\top Q_N x_N$ . If  $J_{k+1}^*(x_{k+1}) = x_{k+1}^\top P_{k+1} x_{k+1}$  with  $P_{k+1} \geq 0$ , the quantity minimized in the Bellman recursion is

$$\begin{aligned} & x_k^\top Q_k x_k + u_k^\top R_k u_k + (A_k x_k + B_k u_k)^\top P_{k+1} (A_k x_k + B_k u_k) \\ &= x_k^\top (Q_k + A_k^\top P_{k+1} A_k) x_k + 2u_k^\top B_k^\top P_{k+1} A_k x_k + u_k^\top (R_k + B_k^\top P_{k+1} B_k) u_k. \end{aligned}$$

Because  $R_k > 0$  and  $P_{k+1} \geq 0$ , the control Hessian satisfies  $R_k + B_k^\top P_{k+1} B_k > 0$ . Completing the square therefore gives a unique linear minimizing feedback law. Substituting that law back into the expression leaves another quadratic function of  $x_k$ , completing the induction and yielding

$$K_k = (R_k + B_k^\top P_{k+1} B_k)^{-1} B_k^\top P_{k+1} A_k, \quad (2.14)$$

$$u_k^* = \mu_k^*(x_k) = -K_k x_k, \quad (2.15)$$

$$P_k = Q_k + A_k^\top P_{k+1} A_k - A_k^\top P_{k+1} B_k (R_k + B_k^\top P_{k+1} B_k)^{-1} B_k^\top P_{k+1} A_k, \quad P_N = Q_N. \quad (2.16)$$

The corresponding optimal cost-to-go is  $J_k^*(x_k) = x_k^\top P_k x_k$ . Since the minimized cost is nonnegative,  $P_k \geq 0$ , which closes the induction. Thus a continuous-state, continuous-control problem can have an exact analytic DP solution. Chapter 5 develops the Riccati geometry, constraints, and MPC connections in detail.

## 2.3 Stochastic Finite-Horizon Dynamic Programming

In the deterministic problem, a state–control pair fixes one successor. A stochastic problem must additionally account for the distribution of possible successors and costs. Write

$$x_{k+1} = f_k(x_k, u_k, w_k), \quad k = 0, \dots, N-1. \quad (2.17)$$

where  $w_k \in \mathcal{W}_k$  is the disturbance at stage  $k$ . In control terminology,  $f_k$  is the system dynamics; together with the conditional law of  $w_k$ , it specifies the stochastic transition model. The controller selects  $u_k$  after observing  $x_k$  but before observing  $w_k$ . Because future disturbances and states are not yet known, an open-loop input sequence is generally insufficient. The decision object is a feedback-policy sequence  $\mu = (\mu_0, \dots, \mu_{N-1})$  with

$$u_k = \mu_k(x_k), \quad \mu_k(x_k) \in \mathcal{U}_k(x_k) \quad \text{for every } x_k \in \mathcal{X}_k. \quad (2.18)$$

A rule that uses  $w_k$  before it is revealed solves a different information pattern.

Let  $h_k = (x_0, u_0, \dots, x_k)$  denote the observed history. The controlled Markov property is an additional statement about the disturbance law, not a consequence of Equation (2.17) alone: an unrecorded road condition, for example, could make the law of  $w_k$  depend on earlier history. We assume that the conditional law of the next disturbance—and therefore of the next state and cost—depends on the history only through the current state and selected control. For every measurable disturbance set  $D \subseteq \mathcal{W}_k$ ,

$$\mathbb{P}(w_k \in D \mid h_k, u_k) = \mathbb{P}_k^w(D \mid x_k, u_k). \quad (2.19)$$

The subscript permits a time-varying law, so the Markov property does not imply stationarity. Both  $x$  and  $u$  may be discrete or continuous; the expectation formulation below is not tied to a transition table. This property is what permits the future problem to be summarized by the cost-to-go  $J_k^*(x_k)$ . If relevant information from the history is omitted from  $x_k$ , the state must be augmented or the cost-to-go must depend on that history.

Let  $\mathcal{M}_{k:N-1}$  denote the class of admissible Markov feedback sequences from stage  $k$  onward. The expected cost of  $\mu \in \mathcal{M}_{0:N-1}$  is

$$J_\mu(x_0) = \mathbb{E}^\mu \left[ \sum_{k=0}^{N-1} g_k(x_k, \mu_k(x_k), w_k) + g_N(x_N) \mid x_0 \right]. \quad (2.20)$$

The superscript in  $\mathbb{E}^\mu$  indicates that the trajectory distribution is induced by applying the feedback sequence  $\mu$  to the dynamics and conditional disturbance laws. The optimal value of the original problem and its stage- $k$  optimal costs-to-go are

$$J^*(x_0) := \inf_{\mu \in \mathcal{M}_{0:N-1}} J_\mu(x_0), \quad (2.21)$$

$$J_k^*(x) := \inf_{\mu_{k:N-1} \in \mathcal{M}_{k:N-1}} \mathbb{E}^{\mu_{k:N-1}} \left[ \sum_{t=k}^{N-1} g_t(x_t, \mu_t(x_t), w_t) + g_N(x_N) \mid x_k = x \right]. \quad (2.22)$$

Thus  $J^*(x_0) = J_0^*(x_0)$ . The infimum accommodates a general formulation; to write the recursion below with attained minima and deterministic Markov selectors, we use the following well-posedness assumption.

**Assumption 2.2** (Well-posed stochastic finite-horizon problem). The controlled Markov property holds and each  $\mathcal{U}_k(x_k)$  is nonempty. For every admissible  $(x_k, u_k)$ ,  $f_k(x_k, u_k, w_k) \in \mathcal{X}_{k+1}$  almost surely under  $\mathbf{P}_k^w(\cdot \mid x_k, u_k)$ . All conditional expectations required by the Bellman recursion are well defined, every displayed minimum is attained by an admissible measurable selector, and the prescribed initial state admits at least one policy of finite expected cost. Finite state and control sets with finite costs are a sufficient, but not necessary, setting.

The first-control/remaining-tail decomposition used in the deterministic case extends to the stochastic problem by conditioning on the current state and control and then averaging over the possible successors. The proposition states why stochastic backward DP is correct; Algorithm 2.2 then states how to execute the recursion (Bertsekas 2019, Sec. 1.2).

**Proposition 2.3** (Stochastic finite-horizon Bellman recursion). *Under assumption 2.2, the terminal condition is*

$$J_N^*(x_N) = g_N(x_N), \quad \text{for every } x_N \in \mathcal{X}_N. \quad (2.23)$$

For  $k = N - 1, \dots, 0$  and every  $x_k \in \mathcal{X}_k$ , the optimal value satisfies

$$J_k^*(x_k) = \min_{u_k \in \mathcal{U}_k(x_k)} \mathbb{E}_{w_k \mid x_k, u_k} [g_k(x_k, u_k, w_k) + J_{k+1}^*(f_k(x_k, u_k, w_k))]. \quad (2.24)$$

Here  $\mathbb{E}_{w_k \mid x_k, u_k}$  denotes expectation under  $\mathbf{P}_k^w(dw \mid x_k, u_k)$ . For every  $x_k \in \mathcal{X}_k$ , any measurable selector satisfying

$$\mu_k^*(x_k) \in \arg \min_{u_k \in \mathcal{U}_k(x_k)} \mathbb{E}_{w_k \mid x_k, u_k} [g_k(x_k, u_k, w_k) + J_{k+1}^*(f_k(x_k, u_k, w_k))] \quad (2.25)$$

defines an optimal deterministic Markov feedback sequence  $\mu^* = (\mu_0^*, \dots, \mu_{N-1}^*)$ , and  $J_0^*(x_0) = J^*(x_0)$ .

*Proof.* At  $k = N$ , no decision remains, so  $J_N^*(x_N) = g_N(x_N)$ . Fix  $k < N$ , an arbitrary  $x_k \in \mathcal{X}_k$ , and a feasible current control  $u_k$ . After  $w_k$  is realized, the successor is  $x_{k+1} = f_k(x_k, u_k, w_k)$ . By the controlled Markov property, the optimal conditional cost of all subsequent stages depends on the earlier history only through this successor and equals  $J_{k+1}^*(x_{k+1})$ . The tower property therefore gives the candidate expected cost associated with  $u_k$  as

$$\mathbb{E}_{w_k \mid x_k, u_k} [g_k(x_k, u_k, w_k) + J_{k+1}^*(f_k(x_k, u_k, w_k))].$$

Minimizing this quantity over the feasible current controls proves Equation (2.24); selecting a minimizer proves Equation (2.25). Backward induction from  $N$  establishes the result at every stage, and at  $k = 0$  the tail problem is the original problem, so  $J_0^*(x_0) = J^*(x_0)$ .  $\square$

**Algorithm 2.2: Stochastic finite-horizon DP**

For a finite-state implementation in which every conditional expectation and feasible-control minimization can be evaluated exactly:

1. **Backward initialization:** set  $J_N^*(x_N) = g_N(x_N)$  for every  $x_N \in \mathcal{X}_N$ .
2. **Backward recursion:** for  $k = N - 1, \dots, 0$ , and for every  $x_k \in \mathcal{X}_k$ , compute

$$J_k^*(x_k) = \min_{u_k \in \mathcal{U}_k(x_k)} \mathbb{E}_{w_k|x_k, u_k} [g_k(x_k, u_k, w_k) + J_{k+1}^*(f_k(x_k, u_k, w_k))].$$

3. **Forward initialization:** for the prescribed initial state, compute

$$u_0^* \in \arg \min_{u_0 \in \mathcal{U}_0(x_0)} \mathbb{E}_{w_0|x_0, u_0} [g_0(x_0, u_0, w_0) + J_1^*(f_0(x_0, u_0, w_0))].$$

After  $u_0^*$  is applied and  $w_0$  is realized, the system incurs  $g_0(x_0, u_0^*, w_0)$  and transitions to

$$x_1 = f_0(x_0, u_0^*, w_0).$$

4. **Forward continuation:** for  $k = 1, \dots, N - 1$ , observe  $x_k$  and compute

$$u_k^* \in \arg \min_{u_k \in \mathcal{U}_k(x_k)} \mathbb{E}_{w_k|x_k, u_k} [g_k(x_k, u_k, w_k) + J_{k+1}^*(f_k(x_k, u_k, w_k))].$$

The disturbance is then realized, the stage cost is incurred, and

$$x_{k+1} = f_k(x_k, u_k^*, w_k).$$

The next feedback decision is made after  $x_{k+1}$  is observed.

Return  $J_0^*(x_0)$  and the optimal feedback decisions along the realized trajectory.

As in the deterministic case, the forward minimizations recover optimal controls from the stored exact cost-to-go functions. Equivalently, minimizing selectors  $\mu_k^*$  may be stored during the backward pass and applied as  $u_k^* = \mu_k^*(x_k)$ ; evaluating a stored selector is policy execution, not online planning. Re-evaluating the expectation and minimization from the measured state during deployment, while keeping the model and  $J_{k+1}^*$  fixed, is one-step online planning in the restricted computational sense used in this book; resolving a longer tail problem is multistep online planning. With the same exact model, cost-to-go functions, minimization, and tie-breaking rule, the stored selector and one-step calculation return the same optimal control: the online calculation recovers rather than improves the exact DP policy. The exact backward pass requires the conditional disturbance law, or an equivalent transition-and-cost model, for every represented state and feasible current control; it is therefore an explicit-model DP computation.

The system-and-disturbance form above is primary. For a finite next-state space, the same transition model can be represented by the pointwise probabilities

$$p_k(\xi | x_k, u_k) = \mathbb{P}(x_{k+1} = \xi | x_k, u_k), \quad \xi \in \mathcal{X}_{k+1}. \quad (2.26)$$

Unlike the deterministic model, which has one successor  $f_k(x_k, u_k)$ , the probabilities  $p_k$  describe how likely the possible successors are. Define also

$$\bar{g}_k(x_k, u_k) = \mathbb{E}_{w_k|x_k, u_k}[g_k(x_k, u_k, w_k)]. \quad (2.27)$$

Then the finite-state specialization of the same recursion is

$$J_k^*(x_k) = \min_{u_k \in \mathcal{U}_k(x_k)} \left\{ \bar{g}_k(x_k, u_k) + \sum_{\xi \in \mathcal{X}_{k+1}} p_k(\xi | x_k, u_k) J_{k+1}^*(\xi) \right\}. \quad (2.28)$$

The expected stage cost  $\bar{g}_k$  remains necessary when  $g_k$  depends directly on  $w_k$ . For a continuous next-state space, one uses the expectation form in Equation (2.24) rather than this finite probability sum. Thus the transition table is a useful finite-state computational specialization, not the definition of stochastic DP.

The stochastic exact recursion has the parallel Q-factor form

$$Q_k^*(x_k, u_k) = \mathbb{E}_{w_k|x_k, u_k}[g_k(x_k, u_k, w_k) + J_{k+1}^*(f_k(x_k, u_k, w_k))], \quad (2.29)$$

$$\begin{aligned} J_k^*(x_k) &= \min_{u_k \in \mathcal{U}_k(x_k)} Q_k^*(x_k, u_k), \\ \mu_k^*(x_k) &\in \arg \min_{u_k \in \mathcal{U}_k(x_k)} Q_k^*(x_k, u_k). \end{aligned} \quad (2.30)$$

The Q-factor is defined for  $k = 0, \dots, N-1$  and every admissible  $(x_k, u_k)$ ; the following two minimization relations hold for every  $x_k \in \mathcal{X}_k$ . At the final decision stage,

$$Q_{N-1}^*(x_{N-1}, u_{N-1}) = \mathbb{E}_{w_{N-1}|x_{N-1}, u_{N-1}}[g_{N-1}(x_{N-1}, u_{N-1}, w_{N-1}) + g_N(f_{N-1}(x_{N-1}, u_{N-1}, w_{N-1}))].$$

There is no next-stage Q-factor or next-control minimization in this terminal initialization. For  $k = 0, \dots, N-2$ , write  $x_{k+1} = f_k(x_k, u_k, w_k)$  inside the expectation. Substituting  $J_{k+1}^*(x_{k+1}) = \min_{v \in \mathcal{U}_{k+1}(x_{k+1})} Q_{k+1}^*(x_{k+1}, v)$  yields

$$Q_k^*(x_k, u_k) = \mathbb{E}_{w_k|x_k, u_k} \left[ g_k(x_k, u_k, w_k) + \min_{v \in \mathcal{U}_{k+1}(x_{k+1})} Q_{k+1}^*(x_{k+1}, v) \right]. \quad (2.31)$$

This recursion remains stochastic: its conditional expectation averages the possible current costs and successor costs-to-go. The Q-factor stores that expected cost for each current state–control pair; it does not replace the stochastic problem by a deterministic one. It should also not be confused with the state-weight matrices  $Q_k$  used in the preceding LQ example and later LQR chapters.

The Q-factor form is especially convenient for sample-based computation and therefore provides a natural route toward model-free value learning. After control  $u_k$  is applied at  $x_k$ , one observed or simulated successor and its realized stage cost make

$$g_k(x_k, u_k, w_k) + \min_{v \in \mathcal{U}_{k+1}(x_{k+1})} Q_{k+1}(x_{k+1}, v)$$

a sample target for the executed pair  $Q_k(x_k, u_k)$ . The update can therefore use a transition sample without analytically evaluating  $f_k$ , the disturbance law, or the conditional expectation, and that one sample need not evaluate current controls that were not applied. When such samples come from the physical environment and the controller does not query an explicit predictive model, this is both a sample-based Bellman computation and a model-free RL architecture. By contrast, a direct Bellman optimality backup for  $J_k(x_k)$  must compare candidate values for the competing current controls; without a model, it requires control-conditioned samples collected across those controls.

The Q-factor itself, however, is not intrinsically model-free: it is a state–control cost representation, not a specification of how its backup is computed. The exact Q recursion above evaluates a conditional expectation from an explicit model and is therefore model-based in the Bellman/DP implementation sense. Conversely, transitions generated by a known or learned simulator can support a sample-based Q update while the overall architecture remains model-based because it uses that predictive model. This is the two-axis distinction introduced in Chapter 1. Either route still requires coverage of the relevant state–control pairs. A stored  $\tilde{Q}_k$  can avoid explicit successor propagation during online control selection, but continuous controls still require the feasible minimization  $\arg \min_{u_k \in \mathcal{U}_k(x_k)} \tilde{Q}_k(x_k, u_k)$ . The later infinite-horizon treatment develops the stationary Q-factor recursion and its connection to Q-learning.

#### Continuous-state and continuous-control example: stochastic LQ control.

Add zero-mean process noise to Equation (2.13):

$$x_{k+1} = A_k x_k + B_k u_k + w_k, \quad \mathbb{E}[w_k | x_k, u_k] = 0, \quad \mathbb{E}[w_k w_k^\top | x_k, u_k] = W_k, \quad (2.32)$$

where  $W_k \geq 0$  is a prescribed, stage-dependent conditional covariance that does not depend on  $(x_k, u_k)$ ; no Gaussian assumption is required. With full-state feedback, unconstrained controls, and the expected quadratic cost associated with Equation (2.13), suppose the next cost-to-go is  $J_{k+1}^*(x_{k+1}) = x_{k+1}^\top P_{k+1} x_{k+1} + c_{k+1}$ . Its expected successor term is

$$\begin{aligned} & \mathbb{E}[(A_k x_k + B_k u_k + w_k)^\top P_{k+1} (A_k x_k + B_k u_k + w_k) | x_k, u_k] + c_{k+1} \\ &= (A_k x_k + B_k u_k)^\top P_{k+1} (A_k x_k + B_k u_k) + \text{tr}(P_{k+1} W_k) + c_{k+1}. \end{aligned}$$

The zero-mean cross terms vanish, and the trace term is independent of  $u_k$ . After the current stage cost  $x_k^\top Q_k x_k + u_k^\top R_k u_k$  is added, the stochastic Bellman objective differs from its deterministic counterpart only by  $\text{tr}(P_{k+1} W_k) + c_{k+1}$ . Completing the square therefore gives the same  $K_k$ ,  $P_k$ , and minimizing feedback law  $u_k^* = \mu_k^*(x_k) = -K_k x_k$ , while backward induction adds a state-independent constant:

$$J_k^*(x_k) = x_k^\top P_k x_k + c_k, \quad c_N = 0, \quad c_k = c_{k+1} + \text{tr}(P_{k+1} W_k). \quad (2.33)$$

The matrices  $P_k$  and gains  $K_k$  are exactly those in Equations (2.14) and (2.16); additive zero-mean noise raises the expected cost by constants but does not change the minimizing feedback law. Here *certainty equivalence* means that replacing the additive disturbance by its conditional mean gives the same optimal feedback law, although not the same optimal expected value. This

is an exact finite-horizon result under the stated assumptions, not a general property of stochastic control. Nonzero-mean, multiplicative, or control-dependent noise and constraints can change the optimal policy; partial observation requires a separate state-estimation and separation argument. Section 2.4 later distinguishes this exact special case from using a nominal or certainty-equivalent model as an approximation when these assumptions do not hold.

## 2.4 From Exact to Approximate Finite-Horizon DP

Exact DP must represent or evaluate every cost-to-go quantity required by its backups; a tabular implementation does so at every represented state and stage. This becomes a computational bottleneck when the state space is large or continuous, the conditional expectation is expensive, the minimization is difficult, or the required quantities are accessible only through data. Approximate DP replaces one or more of these exact objects or computations by tractable surrogates.

Figure 2.5 organizes approximation according to its role in a finite-horizon Bellman decision. The three internal computations that may be approximated are the next-stage cost-to-go, the conditional expectation, and the feasible-control minimization.

The remainder of this section first establishes a common approximate Bellman framework. Its three numbered subsections then follow Figure 2.5 from right to left: approximate cost-to-go, approximate expectation, and approximate minimization.

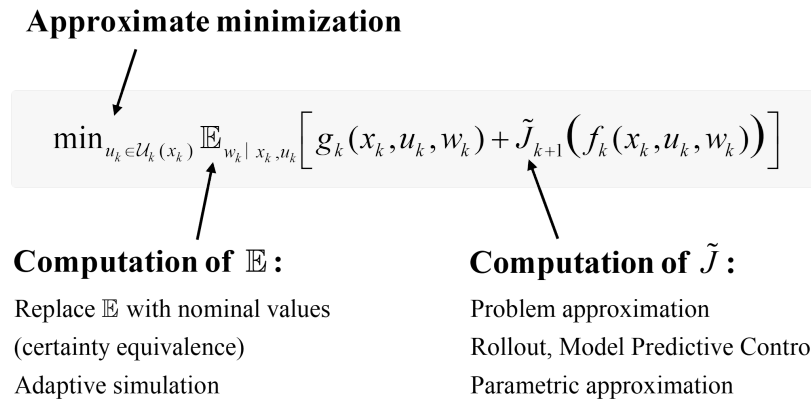


Figure 2.5: Three entry sites for approximation in a finite-horizon Bellman computation: the next-stage cost-to-go, conditional expectation, and feasible-control minimization. Adapted from the value-space organization in Bertsekas (2019, Fig. 2.1.1).

### A common approximate Bellman framework.

Sections 2.2–2.3 constructed each Bellman decision from the exact next-stage cost-to-go  $J_{k+1}^*$ . A basic value-space approximation replaces this unavailable or expensive function by a tractable  $\tilde{J}_{k+1}$ . If the remaining Bellman computations are still exact, the deterministic one-step lookahead law is

$$\tilde{\mu}_k(x_k) \in \arg \min_{u_k \in \mathcal{U}_k(x_k)} \{g_k(x_k, u_k) + \tilde{J}_{k+1}(f_k(x_k, u_k))\}, \quad x_k \in \mathcal{X}_k, \quad (2.34)$$

whereas the stochastic lookahead first forms the Q-factor induced by  $\tilde{J}_{k+1}$ ,

$$Q_k^{\tilde{J}}(x_k, u_k) = \mathbb{E}_{w_k|x_k, u_k} [g_k(x_k, u_k, w_k) + \tilde{J}_{k+1}(f_k(x_k, u_k, w_k))], \quad (2.35)$$

$$\tilde{\mu}_k(x_k) \in \arg \min_{u_k \in \mathcal{U}_k(x_k)} Q_k^{\tilde{J}}(x_k, u_k), \quad x_k \in \mathcal{X}_k. \quad (2.36)$$

These equations isolate the cost-to-go approximation. More generally, the conditional expectation in Equation (2.35) may be replaced by a numerical or sample-based estimate, and the exact arg min in Equations (2.34) and (2.36) may be replaced by a feasible approximate solution. These are separate approximation choices: approximating one site does not require approximating the other two. For a stochastic problem, the information pattern must still be respected: the expected cost of each current control is evaluated before the controls are compared.

Approximate finite-horizon DP may also construct its cost-to-go surrogates backward. Starting from a terminal-cost representation, normally exact because  $g_N$  is known,

$$\tilde{J}_N(x_N) = g_N(x_N), \quad x_N \in \mathcal{X}_N, \quad (2.37)$$

one may construct, at selected states and for  $k = N - 1, \dots, 0$ , targets of the form

$$\bar{J}_k(x_k) = \min_{u_k \in \mathcal{U}_k(x_k)} \mathbb{E}_{w_k|x_k, u_k} [g_k(x_k, u_k, w_k) + \tilde{J}_{k+1}(f_k(x_k, u_k, w_k))], \quad \tilde{J}_k(x_k) \approx \bar{J}_k(x_k). \quad (2.38)$$

Omitting  $w_k$  and the expectation gives the deterministic recursion in Section 2.2. If the selected function class cannot represent  $g_N$  exactly, the terminal representation must also be fitted and becomes another declared approximation. The relation  $\tilde{J}_k \approx \bar{J}_k$  identifies what must be approximated; it is not yet a complete computational method. The next three subsections explain how each site in the figure can be treated.

#### 2.4.1 Approximate Cost-to-Go

The rightmost site in Figure 2.5 replaces the exact cost of the remaining problem by a tractable evaluator. The evaluator need not come from one particular algorithm. Three important constructions are problem approximation, rollout or MPC, and parametric approximation.

#### Problem approximation.

Before choosing a numerical representation, one may simplify the OCP itself: its dynamics, uncertainty, constraints, decomposition, horizon, or state description. Solving the surrogate

problem can supply an approximate cost-to-go or policy for the original problem. In some applications the surrogate is instead implemented directly, in which case problem approximation replaces the original OCP rather than only its cost-to-go. The mobility case in Section 2.5.2 reduces a long time-grid energy-management problem to a link-based parameter problem and solves that surrogate directly.

### Rollout and MPC.

For a finite-horizon problem, let a feasible stage-dependent base policy be

$$\mu_{\text{base}} = (\mu_{\text{base},0}, \dots, \mu_{\text{base},N-1}).$$

Rollout evaluates the future cost produced by this policy, exactly or by simulation, and uses that base-policy cost as an approximate cost-to-go. A one- or multistep lookahead then compares current controls before following the base policy. A truncated rollout may append a terminal cost-to-go approximation after several base-policy steps.

MPC can be interpreted as a specialized member of the same rollout–lookahead family, but its operational mechanism is different. Rather than merely evaluating a preselected base policy, MPC reoptimizes a moving-horizon OCP, applies its first control, and rebuilds the problem at the next state. The shortened horizon and its terminal cost or constraint approximate the omitted tail whenever the moving-horizon problem is shorter than the original remaining problem. The feedback law induced by MPC can also serve as a rollout base policy. Chapters 4–5 develop these connections and the role of online improvement.

### Parametric approximation.

A parameterized evaluator such as  $\tilde{J}_k(x_k; r_{J,k})$  can compress and interpolate cost-to-go information available only at selected states. Its fitting targets may be the approximate Bellman values introduced in Equation (2.38), or values obtained from a surrogate problem, rollout, or sampled experience. Thus parametric approximation is a way to represent and reuse the approximate cost-to-go, rather than a fourth Bellman-computation site. For a finite horizon, the stage dependence generally requires separate parameters for different stages or the inclusion of the stage or time-to-go as an input. Detailed backup–fit equations are deferred to the primarily stationary infinite-horizon treatment later in this chapter. Chapter 3 then develops representation and fitting, and Chapter 4 develops executable value-space methods; the same representation idea also applies to Q-factors.

## 2.4.2 Approximate Expectation

The middle site in Figure 2.5 concerns the conditional expectation in a stochastic Bellman computation. Three common approaches are certainty equivalence, scenario or Monte Carlo approximation, and adaptive simulation.

### Certainty equivalence.

Certainty-equivalent control replaces  $w_k$  by a representative value such as  $\bar{w}_k(x_k, u_k) = \mathbb{E}[w_k \mid x_k, u_k]$  and computes

$$\arg \min_{u_k \in \mathcal{U}_k(x_k)} \{g_k(x_k, u_k, \bar{w}_k(x_k, u_k)) + \tilde{J}_{k+1}(f_k(x_k, u_k, \bar{w}_k(x_k, u_k)))\}.$$

In general, substituting a mean before applying nonlinear cost and dynamics is not the same as taking the conditional expectation. The stochastic LQ problem of Section 2.3 is an important exact exception for the optimal feedback law; elsewhere certainty equivalence is a declared approximation.

### Scenario and Monte Carlo approximation.

Scenario averages or Monte Carlo samples retain multiple disturbance outcomes rather than collapsing them to one representative disturbance. For example, a resettable simulator or equivalent data coverage may supply  $L$  conditionally independent disturbance samples  $w_k^{(\ell)}$  for every candidate  $u_k \in \mathcal{U}_k(x_k)$  at a fixed state  $x_k$ . The conditional expected Q-factor induced by  $\tilde{J}_{k+1}$  can then be estimated by

$$\begin{aligned} \hat{Q}_k^{\tilde{J}}(x_k, u_k) &= \frac{1}{L} \sum_{\ell=1}^L [g_k(x_k, u_k, w_k^{(\ell)}) + \tilde{J}_{k+1}(f_k(x_k, u_k, w_k^{(\ell)}))], \\ \hat{\mu}_k(x_k) &\in \arg \min_{u_k \in \mathcal{U}_k(x_k)} \hat{Q}_k^{\tilde{J}}(x_k, u_k). \end{aligned} \quad (2.39)$$

The samples first estimate each candidate control's conditional expected cost; the controls are compared only after these averages have been formed. Minimizing separately within each disturbance sample would instead allow the current control to depend on a disturbance that has not yet been observed and would solve a different information pattern.

### Adaptive simulation.

Instead of assigning the same simulation budget to every candidate, adaptive simulation allocates samples sequentially to branches whose expected values are promising or still uncertain. It therefore approximates the expectation and candidate search jointly (Bertsekas 2019, Sec. 2.4.2).

## 2.4.3 Approximate Minimization

The leftmost site in Figure 2.5 replaces the exact feasible-control minimization by a tractable decision procedure.

### Candidate restriction or discretization.

The feasible-control minimum may be approximated by searching a finite grid or a smaller candidate subset. This reduces computation but can miss a better control outside the restricted set.

**Inexact numerical optimization.**

For continuous or high-dimensional controls, a local solver or an early-terminated iterative solver may replace global exact minimization. Its returned control must remain in  $\mathcal{U}_k(x_k)$ ; feasibility may require a feasible parameterization, projection, or a separate constraint-handling mechanism.

**Policy proposals and direct policies.**

A fitted policy can propose a candidate or warm-start the numerical optimization. It may instead output a control directly and bypass the online minimization. The latter case is more naturally classified as approximation in policy space than as an approximate numerical minimizer: approximation quality and constraint handling have moved into policy construction and execution. Chapter 6 develops that distinction.

**Combining the three sites.**

These routes can be combined. For example, a learned terminal cost-to-go approximation may close a short MPC problem, sampled scenarios may approximate its expectation, and a fitted policy may warm-start its optimization. Here  $g_N$  remains the terminal cost of the original finite-horizon problem; *terminal cost-to-go approximation* refers specifically to an evaluator placed at the artificial end of a shortened lookahead horizon.

**Approximation and online execution.**

The location of approximation does not by itself determine the online architecture. A stored  $\tilde{J}$  evaluates the remaining cost, but its Bellman decision still needs an online model or simulator and feasible-control minimization. A stored  $\tilde{Q}$  avoids online propagation through a successor model, but retains the feasible-control minimization. A fitted policy can act directly, while transferring approximation and feasibility requirements to policy construction. Model use during target construction should therefore be distinguished from model use during online control selection.

## 2.5 Mobility Case: HEV Energy Management

HEV energy management allocates a requested traction power among two or more onboard energy sources while respecting their dynamic and operating limits. This allocation—the *power split*—is a central decision within the broader supervisory control problem. Because present battery use changes the energy available later, an optimal decision must balance current conversion loss against future operating opportunities and constraints.

A physically meaningful finite-horizon energy-management problem exposes both difficulties introduced in Chapter 1. It can contain thousands of coupled decisions, nonlinear component models, and state constraints, creating a computational difficulty. It also depends on future traction demand that is not known exactly when the control is applied, creating an informational difficulty. This section first formulates that problem and then shows how a road-link problem approximation can reduce both burdens. A separate finite-grid model is subsequently used to

show, step by step, how exact backward DP constructs a cost-to-go and a feedback law. The two models share the same energy-allocation structure, but one is not the next numerical stage of the other.

### 2.5.1 Finite-Horizon Formulation of HEV Energy Management

In a two-source HEV, the primary source and battery together meet the power demand created by the intended speed, acceleration, road grade, and auxiliary loads. Drawing more battery power can reduce the primary-source load now, but it changes the stored energy available later. Energy management is therefore an inherently dynamic allocation problem rather than a sequence of unrelated instantaneous choices.

Figure 2.6 shows the common physical structure. The energy-management controller receives the current battery energy state  $x(t)$  and the requested power  $P_{\text{dem}}(t)$  and selects the primary-source power  $u(t)$ . Let  $P_b(t)$  denote battery power, taken as positive when the battery supplies the powertrain. The two sources satisfy  $u(t) + P_b(t) = P_{\text{dem}}(t)$ , while the battery state is written abstractly as  $\dot{x}(t) = F(x(t), u(t), P_{\text{dem}}(t))$ . Here  $F$  represents the battery conversion dynamics after eliminating  $P_b(t)$  through the power balance; conversion losses and operating limits enter through the component models and constraints. The primary source may be an engine or a fuel cell, and its energy or fuel use supplies the running cost. To keep the signal and power flows legible, the balance and state equations are stated here rather than repeated inside the figure.

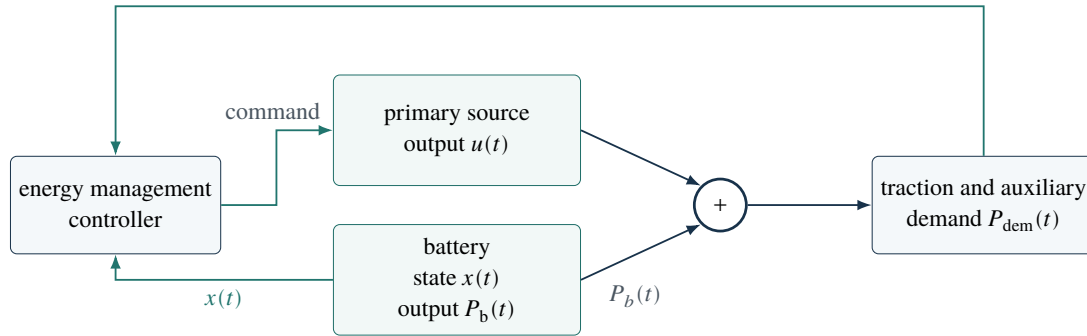


Figure 2.6: Conceptual two-source HEV energy-management architecture. The controller selects the primary-source command  $u(t)$  using the battery state  $x(t)$  and power demand  $P_{\text{dem}}(t)$ ; the battery supplies the remaining power subject to conversion losses and operating constraints. The diagram covers both engine–battery and fuel-cell–battery architectures.

To make the mathematical example auditable, consider the optimal energy-management problem of Choi et al. (2024), instantiated there by a fuel-cell/battery vehicle. The deterministic

finite-horizon benchmark is

$$\begin{aligned}
 & \min_{u(\cdot)} \int_{t_0}^{t_f} \dot{m}_{fc}(u(t)) \, dt \\
 & \text{subject to } \dot{x}(t) = F(x(t), u(t), P_{\text{dem}}(t)), \\
 & \quad \underline{u} \leq u(t) \leq \bar{u}, \quad \underline{x} \leq x(t) \leq \bar{x}, \\
 & \quad x(t_0) = x_0, \quad x(t_f) = x_f.
 \end{aligned} \tag{2.40}$$

Here  $x(t)$  is battery state of energy (SOE),  $u(t) = P_{fc}(t)$  is fuel-cell power,  $P_{\text{dem}}(t)$  is the power demanded from the hybrid powertrain,  $\dot{m}_{fc}$  is hydrogen-consumption rate, and  $F$  is the nonlinear SOE model after enforcing the power balance. The terminal equality makes the comparison charge sustaining when  $x_f = x_0$ .

Solving Equation (2.40) at the initial time requires the complete future function  $P_{\text{dem}}(t)$  over  $[t_0, t_f]$ , rather than only its current value. This full demand preview defines a useful deterministic benchmark, but it is not normally measured in advance. In operation it must be estimated from route, traffic, signal, grade, and intended-motion information; prediction errors would require a stochastic or robust extension.

Two difficulties are already visible. *Computationally*, the horizon is typically tens of minutes, so even a one-second numerical discretization creates thousands of coupled control decisions and state constraints; the nonlinear powertrain model can also make the resulting trajectory optimization nonconvex. *Informationally*, the required future  $P_{\text{dem}}(t)$  depends on future motion, traffic, signals, and road grade. It is not measured at the current time and must instead be forecast or communicated. Consequently, there is no generally available exact real-time solution for arbitrary models, constraints, and future driving conditions.

The energy-allocation objective and decision are vehicle-level, while the battery and fuel-cell models are component-level. Route and traffic information provided through intelligent transportation systems (ITS) and vehicle-to-everything (V2X) communication can expand the information architecture to the mobility-system level. Such connectivity is an enabler for obtaining future information, not part of the mathematical definition of energy management itself. An onboard-only formulation remains vehicle-level.

The next subsection changes the representation of Equation (2.40): it compresses a long time-domain trajectory into a small collection of road-link variables and summaries. The following subsection then makes a separate, more severe simplification solely to expose exact finite-state DP.

### 2.5.2 Link-Domain Problem Approximation for Energy Management

A road network is conventionally represented by nodes and links. A *road link* is the road segment between two neighboring nodes, where a node may mark an intersection, a route endpoint, or another meaningful boundary. Intelligent transportation systems (ITS) routinely organize route, grade, speed, traffic, and travel-time information at this link level. The link is therefore a natural unit for compressing a long energy-management trajectory; connectivity and prediction can supply the link information in operation, but they are enablers rather than part of the approximation itself (Choi et al. 2024).

Figure 2.7 introduces the reformulation before the equations. The upper axis represents the original time-resolved OCP: several time samples of demand, control, and battery state lie inside a single road link. The lower axis retains the link boundaries but replaces those many samples by a demand summary  $\Theta_\ell$ . Write  $u_\ell(t)$  for the restriction of the primary-source control  $u(t)$  to link  $\ell$ . The reduced decision  $\lambda_\ell$  is not an average of  $u_\ell(t)$ ; it parameterizes the within-link control reconstructed from the demand information. Together,  $\lambda_\ell$  and  $\Theta_\ell$  determine the link contribution  $\mathcal{C}_\ell$  to the reduced objective and the net SOE change  $\Gamma_\ell = x_\ell - x_{\ell-1}$ . Here  $n_0, \dots, n_L$  denote the route-boundary nodes and  $t_0, \dots, t_L$  their corresponding passage times.

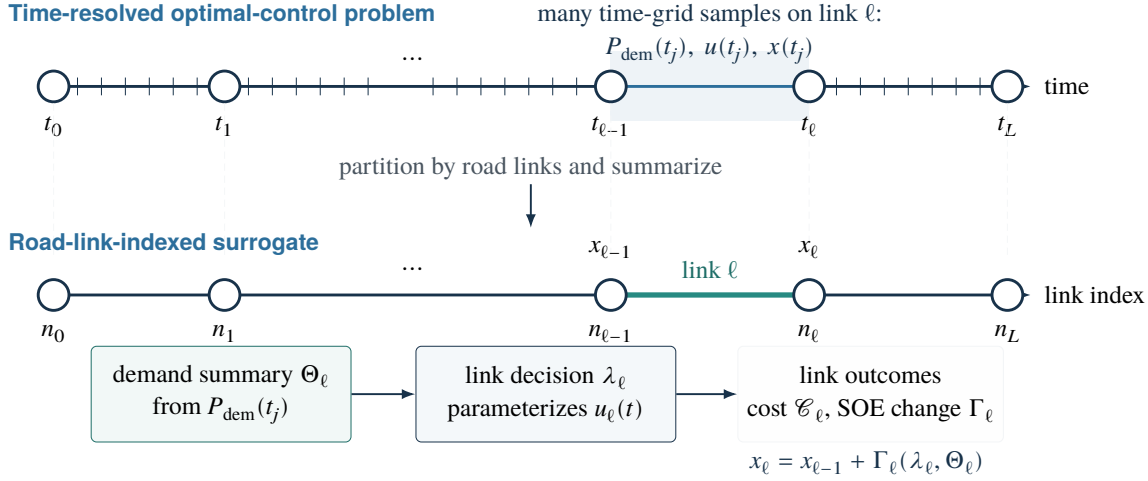


Figure 2.7: Time-resolved and road-link-indexed representations of the energy-management problem. Fine-grid demand samples are compressed into  $\Theta_\ell$ ; the link decision  $\lambda_\ell$  parameterizes the within-link control. Together they determine the link cost  $\mathcal{C}_\ell$  and SOE increment  $\Gamma_\ell$ , which updates the boundary state. Conceptually adapted from Choi et al. (2024, Figs. 1 and 2).

### From a time grid to road links.

Partition the route into  $L$  road links with boundary times  $t_0 < t_1 < \dots < t_L = t_f$  and retain the SOE only at the link boundaries. Regrouping the additive trajectory cost by link is only a change of indexing; the substantive approximation begins when the slowly varying battery state is treated as locally constant while the within-link control is determined:

$$x(t) \approx x_{\ell-1}, \quad t_{\ell-1} < t \leq t_\ell, \quad \ell = 1, \dots, L, \quad (2.41)$$

where  $x_{\ell-1} = x(t_{\ell-1})$ . The state may still change from one link boundary to the next. The demand trajectory within link  $\ell$  is compressed into

$$\Theta_\ell = [E_{\text{dem},\ell}^+ \quad E_{\text{dem},\ell}^- \quad \Delta t_\ell \quad \Delta t_\ell^+]^\top, \quad (2.42)$$

Here  $E_{\text{dem},\ell}^+$  and  $E_{\text{dem},\ell}^- \leq 0$  are the integrals of the positive and negative portions of  $P_{\text{dem}}(t)$ , respectively;  $\Delta t_\ell = t_\ell - t_{\ell-1}$  is the link traversal time; and  $\Delta t_\ell^+$  is the time for which the demand

is positive. ITS or V2X information may supply these summaries, but the approximation itself does not depend on a particular prediction mechanism.

In the construction of Choi et al. (2024), an approximately constant link costate  $\lambda_\ell$ —the marginal cost associated with battery energy—parameterizes the within-link control as  $u_\ell(t) = h(\lambda_\ell, P_{\text{dem}}(t))$ . Evaluating this control gives a link cost  $\mathcal{C}_\ell(\lambda_\ell, \Theta_\ell)$  and an SOE increment  $\Gamma_\ell(\lambda_\ell, \Theta_\ell) = x_\ell - x_{\ell-1}$ . Thus the original time sequence of controls and states is replaced by one decision and one state update per road link.

### From the link surrogate to a QP.

The linkwise reduction alone does not make the problem quadratic or convex. The cited study additionally uses a quadratic control-oriented fuel-source model, a control-oriented battery-loss model, and a mean-demand approximation when constructing the admissible costate bounds. Under these modeling conditions, each link cost is quadratic in  $\lambda_\ell$ , each SOE increment is affine in  $\lambda_\ell$ , and the final problem can be stacked as the convex QP

$$\begin{aligned} \min_{\lambda} \quad & \lambda^\top H(\Theta) \lambda \\ \text{subject to} \quad & A_{\text{eq}}(\Theta) \lambda = b_{\text{eq}}(\Theta, x_0, x_f), \\ & A(\Theta) \lambda \leq b(\Theta, x_0, \underline{x}, \bar{x}), \\ & \underline{\lambda}(\Theta) \leq \lambda \leq \bar{\lambda}(\Theta), \end{aligned} \tag{2.43}$$

where  $\lambda = [\lambda_1, \dots, \lambda_L]^\top$ ,  $\Theta = (\Theta_1, \dots, \Theta_L)$ , and  $H(\Theta) \geq 0$ . The equality enforces the terminal SOE, the linear inequalities enforce the retained link-boundary SOE limits, and the box constraints ensure that the reconstructed within-link power split is admissible. The matrices are obtained from the particular control-oriented component models; they are not universal HEV parameters.

### What the approximation buys.

Table 2.1 compares the two endpoints of this sequence: the original time-resolved OCP and the final link-based QP.

Table 2.1: Original OCP and final link-based QP in the reported 55-link case of Choi et al. (2024).

| Criterion                          | Original time-domain OCP                                                    | Final link-based QP                                                                                  |
|------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Decision dimension</b>          | One control per time sample; typically thousands for a tens-of-minutes trip | One $\lambda_\ell$ per road link; $L = 55$ in the reported case                                      |
| <b>Mathematical structure</b>      | Nonlinear and generally nonconvex trajectory optimization                   | Convex QP after the stated control-oriented approximations                                           |
| <b>Required future information</b> | The complete trajectory $P_{\text{dem}}(t)$                                 | Four aggregate quantities in $\Theta_\ell$ for each link                                             |
| <b>Reported 55-link comparison</b> | Offline DP benchmark                                                        | 0.14% loss relative to DP; at most 2 ms controller time with perfect summaries, excluding prediction |

The reported case therefore replaces thousands of time-indexed decisions by 55 link decisions while remaining close to the offline DP benchmark. The 0.14% loss of optimality and 2 ms computation time are case-study results under perfect link summaries, not general guarantees. If the slow-state and control-oriented model assumptions are inaccurate, the QP is an approximation of the original plant OCP even though the QP itself can be solved globally.

The central lesson is not this particular powertrain derivation. It is that problem approximation can exploit route structure and physical time scales to replace a difficult OCP by a much smaller optimization problem that is solved directly, without first fitting a value function. Applying the same pattern to another two-source HEV requires re-deriving its link summaries, component models, constraints, and convexity.

### 2.5.3 A Deliberately Simplified Model for Exact Backward DP

The preceding QP is a direct solution of a problem approximation; it does not display tabular Bellman recursion. We therefore make a separate pedagogical reduction. The physical structure of HEV energy management is standard in treatments such as Onori et al. (2016): present battery use changes future stored energy, while a drive cycle creates stage-dependent traction demand. The normalized engine/battery model below retains only power balance, battery-energy coupling, local limits, and a charge-sustaining terminal constraint. Unit stage duration and idealized conversion are absorbed into the variables, and the quadratic cost is illustrative rather than a calibrated fuel map. It is a finite-grid teaching surrogate, not a numerical reproduction of the fuel-cell case above.

The example uses an engine/battery HEV so that its two-source interpretation is familiar. Relative to Figure 2.6,  $d_k$  is the sampled counterpart of  $P_{\text{dem}}(t)$ ,  $b_k$  is the chosen power-split control,  $p_k^{\text{eng}} = d_k - b_k$  is the primary-source power, and  $e_k$  is the normalized battery state corresponding to  $x(t)$ . A fuel-cell version would replace engine power and fuel cost by fuel-cell power and hydrogen consumption and would rederive the corresponding battery model and constraints. The reported optimum below is exact only for the declared finite grids and is not a production-vehicle fuel-economy prediction.

#### Mobility case: Charge-sustaining HEV energy management

Let  $e_k$  be battery energy,  $d_k$  the known traction demand,  $b_k$  battery power, and  $p_k^{\text{eng}}$  engine power. Positive  $b_k$  discharges the battery. Because the unit stage duration and conversion factors are absorbed into the normalization, the battery-power command also gives the stage energy change in the following model:

$$p_k^{\text{eng}} = d_k - b_k, \quad (2.44)$$

$$e_{k+1} = e_k - b_k. \quad (2.45)$$

The finite grids and constraints are

$$e_k \in \{0, 1, \dots, 20\}, \quad b_k \in \{-2, -1, 0, 1, 2\}, \quad 0 \leq p_k^{\text{eng}} \leq 8. \quad (2.46)$$

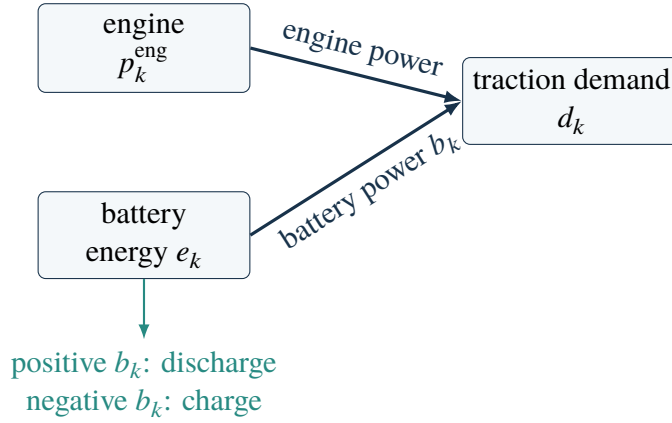


Figure 2.8: Power-flow interpretation of the finite-state HEV example. The known traction demand is split between engine and battery power. Battery power is a control because it changes both the current engine load and the energy available at later stages.

For a 40-stage demand profile, set  $N = 40$ . The stage cost is

$$g_k(e_k, b_k) = 0.06(p_k^{\text{eng}})^2 + 0.30p_k^{\text{eng}} + 0.015b_k^2. \quad (2.47)$$

The initial and terminal energies are both 10. The exact terminal requirement is represented by

$$g_N(e) = \begin{cases} 0, & e = 10, \\ +\infty, & e \neq 10. \end{cases} \quad (2.48)$$

Accordingly, the declared finite problem is

$$\min_{b_0, \dots, b_{N-1}} \left\{ \sum_{k=0}^{N-1} g_k(e_k, b_k) + g_N(e_N) \right\} \quad \text{subject to} \quad (2.44)\text{--}(2.46), \quad e_0 = 10. \quad (2.49)$$

The known sequence  $d_0, \dots, d_{N-1}$  plays the role of a prescribed drive cycle. For reproducibility, the 40 demand values used below are, read row by row,

$$\begin{bmatrix} 1 & 1 & 2 & 2 & 3 & 4 & 5 & 4 & 3 & 2 \\ 1 & 1 & 2 & 3 & 5 & 5 & 4 & 2 & 1 & 1 \\ 2 & 3 & 4 & 5 & 5 & 4 & 3 & 2 & 1 & 2 \\ 3 & 4 & 5 & 5 & 4 & 5 & 4 & 5 & 4 & 4 \end{bmatrix}. \quad (2.50)$$

The constraints in Equation (2.46) are local: they must hold at each stage. The equality  $e_N = 10$  is global because a battery decision early in the cycle affects whether the final target remains reachable. It prevents a controller from reporting low fuel use simply by depleting the battery.

The teaching model intentionally omits calibrated efficiency and fuel maps, engine on/off and gear decisions, voltage, current, and thermal constraints, vehicle longitudinal dynamics, and

uncertain demand. Those omissions delimit the numerical claim; they do not affect the role of the example as an audit of finite-state backward DP.

The quadratic engine term makes high engine power relatively expensive. The battery can therefore shift engine effort from high-demand stages to low-demand stages, but it must return to its initial energy. At state  $e$ , the feasible battery-control set is

$$\mathcal{B}_k(e) = \{b \in \{-2, -1, 0, 1, 2\} : e - b \in \{0, \dots, 20\}, 0 \leq d_k - b \leq 8\}. \quad (2.51)$$

In the generic finite-horizon recursion, the physical variables map as  $x_k = e$ ,  $u_k = b$ ,  $f_k(e, b) = e - b$ , and  $\mathcal{U}_k(e) = \mathcal{B}_k(e)$ . Thus algorithm 2.1 becomes

$$J_N^*(e) = g_N(e), \quad (2.52)$$

$$J_k^*(e) = \min_{b \in \mathcal{B}_k(e)} \{g_k(e, b) + J_{k+1}^*(e - b)\}, \quad k = N - 1, \dots, 0, \quad (2.53)$$

$$\mu_k^*(e) \in \arg \min_{b \in \mathcal{B}_k(e)} \{g_k(e, b) + J_{k+1}^*(e - b)\}, \quad \text{for states with } J_k^*(e) < +\infty. \quad (2.54)$$

Thus each backward comparison stores two objects: the minimum remaining cost  $J_k^*(e)$  and a battery control  $\mu_k^*(e)$  that attains it.

This is a model-based calculation in the DP sense. The controller is given the deterministic energy transition, the entire demand sequence, the stage-cost map, and every constraint before the backward pass. It evaluates every feasible state–control successor on the declared grid; none of these quantities is learned from trajectories in this example.

**Worked Example 2.1** (One backward Bellman backup). At the last stage, the demand is  $d_{39} = 4$ . If  $e_{39} = 12$ , terminal feasibility forces  $b_{39} = 2$ , so  $p_{39}^{\text{eng}} = 2$  and

$$J_{39}^*(12) = 0.06(2)^2 + 0.30(2) + 0.015(2)^2 = 0.90.$$

At  $e_{39} = 10$ , the only terminal-feasible control is  $b_{39} = 0$ , giving  $J_{39}^*(10) = 2.16$ . One stage earlier, with  $e_{38} = 10$  and  $d_{38} = 4$ , Bellman’s comparison is

| $b_{38}$ | current cost | $J_{39}^*(e_{39})$ | total |
|----------|--------------|--------------------|-------|
| −2       | 4.020        | 0.900              | 4.920 |
| −1       | 3.015        | 1.455              | 4.470 |
| 0        | 2.160        | 2.160              | 4.320 |
| 1        | 1.455        | 3.015              | 4.470 |
| 2        | 0.900        | 4.020              | 4.920 |

The immediate-cost minimum  $b_{38} = 2$  is not optimal after the cost-to-go is included; the Bellman-optimal control is  $b_{38} = 0$ . This small calculation is the operation performed at every feasible state and stage by the code. The minimum table entry gives  $J_{38}^*(10) = 4.32$ , and its minimizing column stores  $\mu_{38}^*(10) = 0$ .

Figure 2.9 makes the two stored DP objects visible. The finite cost-to-go varies with both stage and battery energy. Later stages contain fewer running-cost terms, but changing demand and

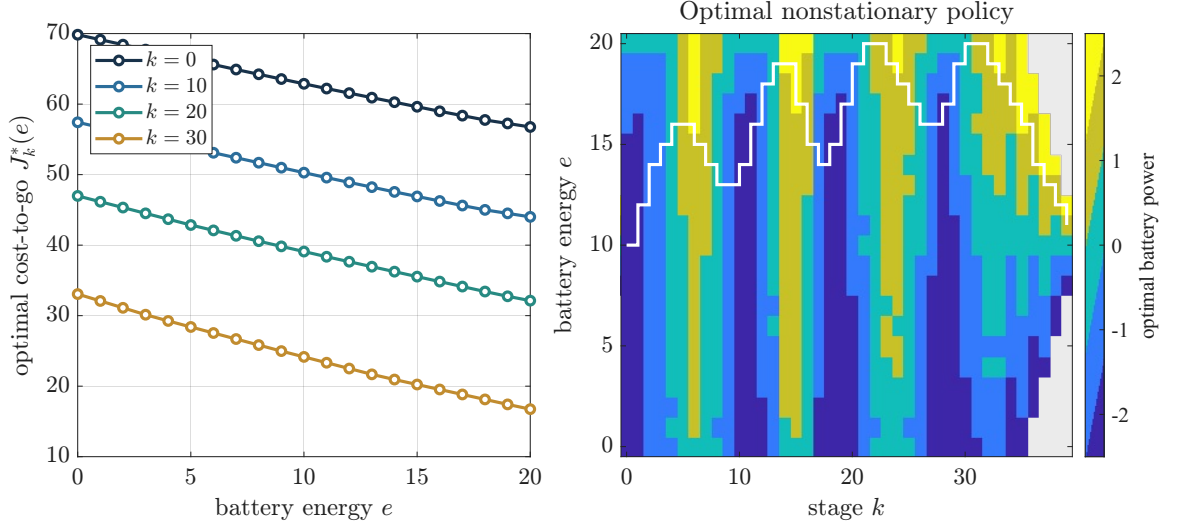


Figure 2.9: Outputs of the exact backward pass. The left panel shows optimal cost-to-go functions at four stages, and the right panel shows the complete nonstationary battery feedback law. The white line is the optimal trajectory from  $e_0 = 10$ ; gray cells are state–stage pairs with no finite-cost, terminal-feasible tail.

terminal reachability mean that monotonicity across  $k$  should not be inferred. The policy map is not a single rule of  $e$  alone: its colors change with  $k$  because both the remaining demand and the distance to the terminal constraint change. This is the nonstationarity expected in a finite-horizon problem.

For comparison, a viability-aware myopic controller minimizes only the current cost in Equation (2.47). It rejects a control if the successor cannot reach the terminal target under the remaining prescribed demand sequence, so the baseline is feasible, but it ignores the magnitude of future cost. It is therefore an oracle-aided diagnostic rather than a causal myopic controller: both policies know the local constraints, future feasibility, and the complete demand sequence, but only DP assigns the full future cost-to-go to the present control.

Table 2.2: Verified results for the normalized finite-state HEV grid. Costs are normalized and should be interpreted only within the declared teaching model.

| Policy                 | Total cost | Terminal energy | Cost relative to myopic |
|------------------------|------------|-----------------|-------------------------|
| Backward DP            | 62.88      | 10              | −12.34%                 |
| Viability-aware myopic | 71.73      | 10              | baseline                |

Table 2.2 and Figure 2.10 report the forward execution of the stored policies. The top panel of the figure compares traction demand with engine power, the middle panel shows the battery control, and the bottom panel shows the resulting stored energy. DP charges during selected lower-demand intervals and discharges during higher-demand intervals, thereby smoothing costly engine peaks while preserving the terminal target. The myopic policy discharges early because

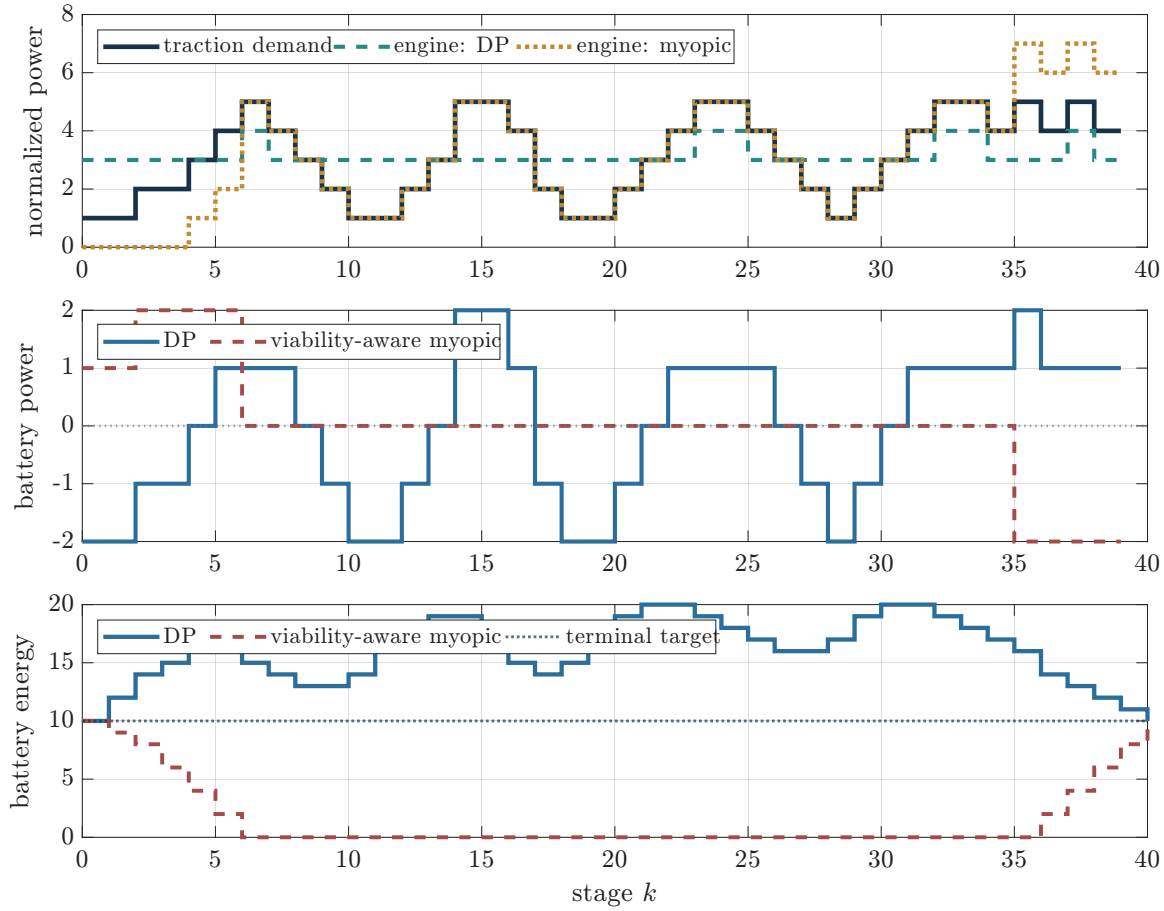


Figure 2.10: Generated numerical results for the declared finite-state HEV model. DP charges during selected lower-demand intervals and discharges during higher-demand intervals. The myopic controller uses battery energy early and is forced to recharge near the end. Both satisfy the same terminal target.

doing so is immediately cheap, remains near its lower energy limit for much of the cycle, and must recharge near the end to recover feasibility. Both policies finish at  $e_N = 10$ , but the DP cost is 12.34% lower within this normalized model. The result measures the value of future-cost information in the declared surrogate; it is not a physical fuel-economy claim.

Both branches in this mobility case remain finite-horizon and stage-dependent: the future-demand description and terminal energy condition distinguish one stage from another. The next teaching unit studies a different class of problems whose model and objective are stationary and whose horizon is infinite.

# Bibliography

- Bertsekas, D. P. (2019). *Reinforcement Learning and Optimal Control*. Belmont, MA: Athena Scientific. ISBN: 9781886529397.
- (2025). *A Course in Reinforcement Learning*. 2nd ed. Nashua, NH: Athena Scientific. ISBN: 9781886529298.
- Choi, K., G. Park, and D. Kum (2024). “An Analytical Approach to the Predictive Energy Management of Connected HEVs: What Information Do We Need to Guarantee Global Optimality?” In: *IEEE Transactions on Intelligent Transportation Systems* 25.9, pp. 12749–12761. DOI: [10.1109/TITS.2024.3384358](https://doi.org/10.1109/TITS.2024.3384358).
- Onori, S., L. Serrao, and G. Rizzoni (2016). *Hybrid Electric Vehicles: Energy Management Strategies*. SpringerBriefs in Electrical and Computer Engineering. London: Springer. DOI: [10.1007/978-1-4471-6781-5](https://doi.org/10.1007/978-1-4471-6781-5).
- Sutton, R. S. and A. G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press. URL: <http://incompleteideas.net/book/the-book-2nd.html>.